UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE ENSINO SUPERIOR DO SERIDÓ
DEPARTAMENTO DE COMPUTAÇÃO E TECNOLOGIA
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

FEATURES ANALYZER: DESENVOLVIMENTO DE UMA PLATAFORMA DE ANÁLISE E VISUALIZAÇÃO DE DADOS

ZAÚ JÚLIO ARAÚJO GALVÃO

Caicó- RN
2022

ZAÚ JÚLIO ARAÚJO GALVÃO

FEATURES ANALYZER: DESENVOLVIMENTO DE UMA PLATAFORMA DE ANÁLISE E VISUALIZAÇÃO DE DADOS

Trabalho de conclusão de curso apresentado ao curso de graduação em Sistemas de Informação, como parte dos requisitos para obtenção do título de Bacharel em Sistemas de Informação da Universidade Federal do Rio Grande do Norte.

Orientador(a): Ms. Luiz Paulo de Assis Barbosa.
.

Caicó- RN
2022

RESUMO

Este trabalho tem como objetivo apresentar uma proposta de desenvolvimento de uma plataforma de análise e visualização de dados extensível e personalizável, visando proporcionar uma forma simples e objetiva de acesso para as bibliotecas de aprendizado de máquina e semelhantes presentes no repositório de bibliotecas *Pypi*. Para atingir este objetivo, foi realizado entrevistas com os pesquisadores da área de Ciência de Dados e Processamento de Sinais, responsáveis pelo projeto de pesquisa *SmartEnergy*, para identificar as principais necessidades e desafios no processo de análise de dados. Além disso, foi realizada uma pesquisa bibliográfica para identificar as principais ferramentas de análise de dados e visualização de dados que possam atender aos requisitos identificados. Por fim, foi proposto um conjunto de bibliotecas e ferramentas para o desenvolvimento da plataforma, que podem ser utilizadas para a criação de um ambiente de desenvolvimento para análise e visualização de dados, que atenda as necessidades do processo de análise de dados.

Palavras-chave: Plataforma, Análise de Dados, Visualização de Dados, Machine Learning, Python.

ABSTRACT

This work aims to present a proposal for the development of an extensible and customizable data analysis and visualization platform, aiming to provide a simple and objective way of accessing machine learning libraries and the like present in the library repository *Pypi* . To achieve this goal, interviews were conducted with researchers in the area of Data Science and Signal Processing, responsible for the research project *SmartEnergy*, to identify the main needs and challenges in the data analysis process. In addition, a literature search was carried out to identify the main data analysis and data visualization tools that can meet the identified requirements. Finally, a set of libraries and tools for the development of the platform was proposed, which can be used to create a development environment for data analysis and visualization, which meets the needs of the data analysis process.

Keywords: Platform, Data Analysis, Data Visualization, Machine Learning, Python.

LISTA DE QUADROS

Quadro 1 – Cronograma de Atividades 18

LISTA DE SIGLAS

ES Engenharia de Software

GUI Graphical User Interface

CSS Cascading Style Sheets

SUMÁRIO

1 INTRODUÇÃO **8**
1.1 Contextualização e Problema **8**
1.2 Objetivos **9**
1.2.1 Objetivo Geral 9
1.2.2 Objetivos Específicos 9
1.3 Delimitação do Estudo **9**
1.4 Justificativa **10**
1.5 Apresentação do Trabalho **10**

2 FUNDAMENTAÇÃO TEÓRICA **12**
2.1 Análise e visualização de dados **12**
2.2 Aprendizado de Máquina **12**
2.3 Engenharia de Software **14**
2.4 Linguagens de Programação **14**
2.5 Bibliotecas **15**
2.5.1 Numpy 15
2.5.2 Pandas 15
2.5.3 Scikit-learn 16
2.5.4 PyQt 16
2.5.5 Seaborn/Matplotlib 16
2.6 Trabalhos Relacionados **16**

3 METODOLOGIA **17**
3.1 Questões de Pesquisa **17**
3.2 Proposta Metodológica **17**
3.3 Atividades **18**
3.3.1 Cronograma de Atividades 18

4 DESENVOLVIMENTO DA PESQUISA **19**
4.1 Levantamento de Requisitos **19**
4.1.1 Requisitos Funcionais 19
4.1.2 Requisitos Não Funcionais 20
4.2 Proposta de Solução **21**
4.2.1 Interfaces de Código 21
4.2.2 Interface Gráfica 21

4.2.2.1 Widgets 22
4.2.2.2 Estilização 22
4.2.2.3 Gráficos 23
4.2.3 Gerenciamento de Dados 23

5 RESULTADOS 25

6 CONCLUSÃO 26
6.1 Discussão 26
6.2 Contribuições 26
6.3 Limitações 27
6.4 Trabalhos Futuros 27

REFERÊNCIAS 28

APÊNDICE A CRONOGRAMA DE ATIVIDADES 29

Capítulo 1
Introdução

A tomada de decisões é um processo que envolve a escolha de uma ou mais alternativas, a partir de um conjunto de opções, para atingir um objetivo. Este processo é realizado por indivíduos, empresas e governos, e é fundamental para o desenvolvimento de qualquer atividade.

Para que uma decisão seja tomada, é necessário que o indivíduo ou grupo de indivíduos tenha conhecimento sobre o problema a ser resolvido, bem como sobre as alternativas que podem ser escolhidas para resolvê-lo. O conhecimento sobre o problema e as alternativas é obtido por informações, que podem ser obtidas de diversas fontes, como sensores, relatórios, entrevistas, gravações, etc.

A partir do conhecimento obtido, o indivíduo ou grupo de indivíduos deve analisar as informações e selecionar as que são mais relevantes para a tomada de decisão. A análise das informações é realizada através de técnicas e ferramentas de análise de dados, utilizadas para extrair conhecimento a partir de dados.

1.1 Contextualização e Problema

A análise de dados é um dos principais fatores para o sucesso de uma empresa, pois é através dela que é possível obter informações que se tornam valiosas para a tomada de decisões. Logo, dado o aumento da quantidade de dados gerados e a necessidade de se obter informações a partir deles, gerou-se o interesse por ferramentas que auxiliem na análise e visualização de dados.

Um grande conjunto de ferramentas já está disponível online, porém, a maioria delas não é gratuita, performática e nem de código aberto. A ausência de ferramentas deste tipo é um fator notável, visto que nem sempre as empresas possuem recursos ou pessoal para desenvolver ferramentas deste tipo.

Nesta perspectiva, surge a oportunidade de se desenvolver uma ferramenta que atenda aos critérios citados anteriormente, e que seja facilmente extensível para novas funcionalidades e diferentes tipos e fontes de dados.

1.2 Objetivos

1.2.1 Objetivo Geral

O objetivo deste trabalho é o estudo e desenvolvimento de um projeto de software com a finalidade de análise e visualização de dados, que seja modular e facilmente extensível, para atender a vários propósitos e fontes de dados.

1.2.2 Objetivos Específicos

- Identificar diferentes problemas de análise de dados, tipos e fontes de dados.
- Modelar um projeto de software que atenda as necessidades identificadas, utilizando os conceitos de Engenharia de Software (ES).
- Implementar o software modelado.
- Testar e validar o software para com aplicação em laboratório.
- Documentar o software para futuras modificações.

1.3 Delimitação do Estudo

As ferramentas de visualização e análise de dados possuem um papel fundamental na tomada de decisão, sendo assim sua utilização é cada vez mais comum em diversas áreas. No entanto, a maioria das ferramentas disponíveis no mercado são proprietárias, de código fechado ou foram criadas em linguagens pouco utilizadas, diminuindo sua acessibilidade e dificulta o uso. Além disso, a maioria das ferramentas disponíveis são específicas para um tipo de problema, dificultando a sua utilização em diversos contextos.

Diante disso, o presente trabalho pretende a extensão de uma ferramente de visualização e análise de dados desenvolvida durante o projeto de pesquisa *Gerenciamento Inteligente do Consumo de Energia*, referido a partir de agora como *SmartEnergy*. O projeto *SmartEnergy* tinha como objetivo a análise de dados de consumo energético em séries temporais. Logo, esse trabalho visa realizar o estudo da capacidade de extensão da ferramenta para outros contextos e evolução dela para uma arquitetura modular e de uso mais amplo.

A ferramenta foi desenvolvida em Python, utilizando a biblioteca Matplotlib para a visualização dos dados. A biblioteca Matplotlib é uma biblioteca de visualização de dados 2D em Python, que produz gráficos de qualidade de publicação em uma variedade de formatos impressos e ambientes interativos em plataformas diversas. A interface gráfica do usuário Graphical User

Interface (GUI) foi desenvolvida utilizando a biblioteca *PyQt* que fornece um conjunto de interfaces para a biblioteca Qt, de código aberto para desenvolvimento de aplicações multiplataforma.

1.4 Justificativa

A tarefa de análise de dados é complexa e exige o conhecimento e emprego de diversas técnicas de processamento. Cada problema requer uma abordagem diferente, e a escolha da técnica mais adequada é um desafio, necessitando de uma vasta gama de ferramentas e conhecimentos.

O processo exploratório da análise de dados requer o emprego de técnicas de visualização de dados, que permitem a compreensão dos dados e a identificação de padrões e tendências. Logo após a visualização dos dados é identificado o problema a ser resolvido, e então a escolha das técnicas que serão utilizadas para a resolução do problema. Frequentemente, a escolha das técnicas é baseada em experiência e intuição, necessitando de uma série de testes e ajustes dos parâmetros para a obtenção de resultados satisfatórios.

Além disso, a aplicação de determinadas técnica são realizadas em conjunto, e a escolha da ordem de aplicação das técnicas é um desafio, pois a ordem de aplicação das técnicas pode influenciar no resultado. Por exemplo, a aplicação de técnicas de redução de dimensionalidade antes da aplicação de técnicas de agrupamento pode resultar em um agrupamento mais eficiente. Logo, existe um número grande de possibilidades de combinação de técnicas, e a escolha da melhor combinação é um desafio.

Desta forma, a proposta deste trabalha é a criação de uma plataforma para análise e visualização de dados, que permita a escolha das técnicas, a combinação e a visualização dos resultados. A plataforma será desenvolvida através do levantamento de requisitos necessários e o desenvolvimento de um protótipo que atenda aos mesmos.

1.5 Apresentação do Trabalho

No capítulo seguinte, fundamentação teórica, é discorrido assuntos relacionados ao trabalho, como Análise e visualização de dados, Aprendizado de Máquina, Engenharia de Software e as linguagens e bibliotecas utilizadas no desenvolvimento do sistema proposto.

No terceiro capítulo é apresentado os tópicos relacionados a modelagem do sistema, contendo os requisitos, funcionais e não funcionais, diagramas, modelo de dados e a arquitetura modular do sistema.

O quarto capítulo é dedicado a implementação do sistema, contendo a descrição do processo de desenvolvimento, a arquitetura do sistema, a descrição das tecnologias utilizadas e a descrição

do processo de desenvolvimento.

Após o desenvolvimento do sistema, no quinto capítulo é apresentado os resultados obtidos com o sistema, contendo a descrição da interface, exemplos de uso e a avaliação do sistema.

Por fim, é apresentado a conclusão obtida com o presente trabalho, bem como sugestões de trabalhos futuros. Seguido das referências bibliográficas utilizadas durante este trabalho, um apêndice com matérias relacionados ao trabalho e um anexo com o código-fonte do sistema.

Capítulo 2
Fundamentação Teórica

2.1 Análise e visualização de dados

A Análise de dados é uma tarefa complexa, mas de fundamental importância para a tomada de decisões. Ela pode ser descrita como um processo de transformação de dados em informações que tenham valor. Com a crescente produção e coleta de dados, pelo surgimento do IOT e dos próprios usuários, esta tarefa tornou-se de suma importância para a evolução da Web como a conhecemos.

O processo de análise de dados pode ser divida em três etapas:

- **Pré-processamento**: como formatação, remoção de dados faltantes e valores atípicos (*Outliers*), normalização, etc. Por exeplo a remoção de *Outliers*, problemas durante a coleta ou arma
- **Processamento**: pode ser realizada de diversas formas, como, por exemplo, a aplicação de métodos estatísticos ou algoritmos de aprendizado de máquina. Por exemplo: regressão, classificação, agrupamento, predição, otimização, entre outros.
- **Pós-processamento**: consiste na apresentação dos resultados obtidos, que pode ser por simples tabelas, gráficos, modelos matemáticos ou até valores numéricos. Por exemplo: visualização de dados em gráficos de séries históricas.

Por fim, a visualização de dados é uma tarefa crucial para a realização de todo o processo, pois é através dela que o usuário realiza a avaliação do processamento para averiguar o emprego dos métodos selecionados e a qualidade dos resultados obtidos.

2.2 Aprendizado de Máquina

O aprendizado de máquina consiste em conjunto de algoritmos visando resolver um problema de forma que algoritmos clássicos não conseguem ou não operam eficientemente. Eles são capazes de gerar modelos matemáticos com base em dados, que podem ser utilizados para prever o comportamento dos dados, agrupar os dados ou classificá-los.

Os problemas que estes algoritmos resolvem podem ser divididos em algumas categorias, como, por exemplo: regressão, classificação, agrupamento, predição, detecção de anomalias, ranqueamento, recomendação, otimização, entre outros.

- **Regressão**: consiste em encontrar uma função que descreve a relação entre uma variável dependente e uma ou mais variáveis independentes. A função encontrada pode ser utilizada para prever o valor da variável dependente para novos valores da variável independente.
- **Classificação**: consiste em encontrar uma função que descreve a relação entre uma variável dependente e uma ou mais variáveis independentes. A função encontrada pode ser utilizada para prever o valor da variável dependente para novos valores da variável independente.
- **Agrupamento**: consiste em encontrar grupos de dados que possuem características semelhantes. Esses grupos podem ser utilizados para realizar análises mais detalhadas sobre os dados.
- **Predição**: consiste em encontrar uma função que descreve a relação entre uma variável dependente e uma ou mais variáveis independentes. A função encontrada pode ser utilizada para prever o valor da variável dependente para novos valores da variável independente.
- **Detecção** de anomalias: consiste em encontrar dados que possuem características diferentes dos demais. Esses dados podem ser utilizados para realizar análises mais detalhadas sobre os dados.
- **Ranqueamento**: consiste em encontrar uma função que descreve a relação entre uma variável dependente e uma ou mais variáveis independentes. A função encontrada pode ser utilizada para prever o valor da variável dependente para novos valores da variável independente.
- **Recomendação**: consiste em encontrar uma função que descreve a relação entre uma variável dependente e uma ou mais variáveis independentes. A função encontrada pode ser utilizada para prever o valor da variável dependente para novos valores da variável independente.
- **Otimização**: consiste em encontrar uma função que descreve a relação entre uma variável dependente e uma ou mais variáveis independentes. A função encontrada pode ser utilizada para prever o valor da variável dependente para novos valores da variável independente.

Cada algoritmo citado, e muitos outros, já estão implementados em bibliotecas de código, algumas de código aberto, como, por exemplo, o *scikit-learn*, e outras de código fechado, como, por exemplo, o MATLAB. Estas bibliotecas são utilizadas para estudo e criação de plataformas para a realização de análises de dados.

2.3 Engenharia de Software

O desenvolvimento de um software é complexo e envolve várias etapas, como o levantamento de requisitos, modelagem do software, desenvolvimento, testes, validão, implementação entre outras. Essas etapas buscam garantir a produção e qualidade do software conforme as necessidades levantadas previamente e validadas.

A engenharia de software surgiu para facilitar e padronizar os projetos e o desenvolvimento de software, para que aumentar eficiência e qualidade do software produzido. Para isso, a engenharia de software utiliza diversas técnicas e ferramentas, como por exemplo: Padrões de Design, Testes, entre outras.

Desenvolvendo algumas dessas técnicas: Padrões de Design, são soluções comuns para vários problemas de projeto, de forma com que possam ser utilizados em contextos diversos. E os Testes, cujo objetivo é garantir a qualidade do software, através da verificação de suas funcionalidades.

2.4 Linguagens de Programação

A linguagem escolhida para esse projeto foi o Python, presente entre as linguagens mais utilizadas no mundo, com uma vasta comunidade e um número crescente de bibliotecas. A linguagem Python é de alto nível, multiparadigma, possui tipagem dinâmica e forte. Além disso, utiliza de uma sintaxe simples e fácil de aprender para pessoas que já possuem conhecimento em outras linguagens de programação, assim como para iniciantes.

A linguagem executa sobre uma máquina virtual, chamada de Python Virtual Machine (*PVM*), responsável por interpretar o código-fonte e executar as instruções. A *PVM* é uma máquina virtual baseada em pilha. Em um software Python o código é compilado para *bytecodes*, uma linguagem intermediária, que a seguir são transpilados, processo de converter o código fonte para outra linguagem intermediária, pela *PVM* código de máquina. O Python usa a compilação *just-in-time* (JIT), que compila o código-fonte para *bytecodes* e depois para código de máquina, em tempo de execução.

O **CPython** é a implementação mais popular da linguagem Python, a implementação oficial da linguagem, e é a implementação que será utilizada nesse projeto. Desenvolvido em C, o **CPython** é multiplataforma, ou seja, pode ser executado em qualquer sistema operacional e arquitetura. O **CPython** funciona na seguinte maneira a seguir:

1. O código-fonte é compilado para *bytecodes*.
2. O *bytecodes* é interpretado pela *PVM*.

3. A *PVM* executa o código.
4. O código é compilado para código de máquina.
5. O código de máquina é executado.

O Python utiliza um sistema de gerenciamento de memória automático, responsável por gerenciar a alocação e desalocação de memória. O Python possui um coletor de lixo, responsável por liberar a memória que não está sendo utilizada. A linguagem trata bem questões relacionadas a paralelismo, concorrência e processamento assíncrono, por bibliotecas como o *asyncio*, a qual é uma biblioteca de código aberto, que implementa o modelo de programação assíncrona.

Em relação à desempenho, a linguagem é relativamente lenta, quando comparado a opções como C e C++, mas possui uma comunidade ativa, com uma vasta gama de bibliotecas e *frameworks*, esse é o principal *trade-off* da linguagem. Logo assim, devido a sua fama, comunidade, rica biblioteca e a alta curva de aprendizado, o Python foi escolhido para o desenvolvimento desse projeto.

2.5 Bibliotecas

Bibliotecas são conjuntos de códigos que podem ser importados para outros projetos, realizando tarefas específicas com resolução já implementadas. A seguir serão apresentadas as principais bibliotecas utilizadas nesse projeto:

2.5.1 Numpy

O Numpy é uma biblioteca fundamental para a computação científica. Ela possui implementações de vetores n-dimensionais em C, compilados executadas de maneira nativa, resultando em alto desempenho. Além disso, o Numpy já dispõe de uma vasta gama de funções matemáticas e operações de álgebra linear. Usando o Numpy é possível realizar, por exemplo, a normalização de um conjunto de dados com apenas uma linha de código.

2.5.2 Pandas

O Pandas, que no que lhe concerne recorre ao Numpy, é uma biblioteca de *open-source*, projetos disponibilizados com código aberto, que estruturam os dados em tabelas (*Dataframes*), estruturas multidimensionais de dados, utilizados para análise de dados. Por utilizar o Numpy, além de uma vasta game de funções matemáticas, o Pandas também possui desempenho de alto nível. Com o Pandas é possível realizar várias operações com os dados, como ordenação, filtragem, entre outros.

2.5.3 Scikit-learn

O Scikit-learn, como o próprio nome sugere, é uma biblioteca de *machine learning*, que possui implementações de algoritmos de aprendizado de máquina, estatísticos e de mineração de dados. O *Scikit-learn* possui conjunto considerável de algoritmos de classificação, regressão, agrupamento, redução de dimensionalidade, entre outros. Além disso, ele possui um grupo de métricas de avaliação, utilizadas para avaliar o desempenho dos modelos criados. Utilizando a biblioteca é possível realizar, por exemplo, a criação de um modelo de árvore de decisão, um exemplo de algoritmo de aprendizado supervisionado, utilizado para classificação e regressão.

2.5.4 PyQt

O PyQt é a biblioteca responsável por criar as interfaces gráficas de usuário (GUI) do projeto. Sobe licença GPL, o PyQt é uma biblioteca de código aberto, que opera por *wrapper's*, funções que encapsulam as bibliotecas pré-compiladas do Qt. O Qt no que lhe concerne, é uma biblioteca, também de código aberto, que dispões de uma série de *widgets*, layouts, entre outros componentes que podem ser facilmente utilizados para a criação de interfaces gráficas. Outra vantagem do PyQt é sua compatibilidade com vários sistemas operacionais, como Linux, Windows, Mac OS, entre outros.

2.5.5 Seaborn/Matplotlib

O Seaborn é a biblioteca gráfica escolhida nesse projeto para tratar a visualização dos dados. Ela é utiliza o Matplotlib como base, fornecendo várias interfaces de alto nível para a criação e personalização de gráficos. O Seaborn possui um conjunto de possibilidades como, por exemplo, a criação de gráficos de dispersão, gráficos de linha, gráficos de barras, entre outros. Utilizando ela é possível embutir os gráficos gerados a interface gráfica criada com o PyQt.

2.6 Trabalhos Relacionados

O principal trabalho relacionado ao nosso é o (GALVãO, 2020). O projeto desenvolvido em 2020 tinha como objetivo a análise de dados de consumo energético, dados providos pela Universidade Federal de Campina Grande - PB. A metodologia do trabalho envolveu o teste de diversos algoritmos de regressão, clusterização e classificação, logo a plataforma proposta aqui seria um agente facilitador considerável no desenvolvimento do projeto (GALVãO, 2020).

Capítulo 3
Metodologia

Neste capítulo, serão apresentados os procedimentos metodológicos utilizados na execução da pesquisa. Primeiramente, serão apresentadas as questões de pesquisa que nortearam a pesquisa. Posteriormente, será apresentada uma proposta metodológica, que descreve os procedimentos utilizados na condução da pesquisa. Ao final, serão apresentadas as atividades realizadas para a realização da pesquisa.

3.1 Questões de Pesquisa

A primeira questão de pesquisa é: *Quais as principais funcionalidades requeridas para realizar a análise de dados?*. Esta questão foi levantada para conseguir identificar todos os requisitos necessários para o desenvolvimento do software proposto. A segunda questão é *Como desenvolver o software que atenda aos requisitos levantados?* e a terceira questão é *Como realizar a avaliação do software?*.

Para responder a primeira questão de pesquisa, foi realizada uma pesquisa de campo com os pesquisadores da área de Ciência de Dados e Processamento de sinais. A pesquisa realizada envolveu uma entrevista com a finalidade de identificar as principais funcionalidades buscadas por estes pesquisadores.

A segunda questão de pesquisa foi respondida com o auxílio da literatura, através da pesquisa bibliográfica e do emprego da engenharia de software a partir dos requisitos previamente levantados.

Já a terceira questão foi respondida com o auxílio dos pesquisadores envolvidos na pesquisa de campo, que avaliaram o software desenvolvido e proporão testes e melhorias para o mesmo.

3.2 Proposta Metodológica

Os pesquisadores selecionados para a pesquisa de campo foram os responsáveis pelo desenvolvimento do projeto *SmartEnergy*, que consiste em um sistema de monitoramento de consumo energia elétrica, com algoritmos de aprendizado de máquina para detecção de padrões e anomalias.

Os pesquisadores foram entrevistados para determinar as principais características que o

software deveria possuir. A partir das respostas obtidas, pode ser determinados os requisitos necessários para o desenvolvimento do software. Com base nesses requisitos, o software proposto pode ser desenvolvido. Os pesquisadores envolvidos levantaram questões como *Qual a melhor forma de visualizar os dados?*, *Como integrar novos algoritmos?*, *Como exportar os resultados?*, *Como obter estatísticas numéricas dos dados?*, *Qual a melhor forma de realizar a importação dos dados?*, entre outras. Todas as questões foram avaliadas e as respostas aceitas foram utilizadas para a definição dos requisitos do software.

A partir dos requisitos levantados, foi realizado a pesquisa bibliográfica para identificar projetos semelhantes e conhecer as soluções implementadas. A partir da pesquisa notamos uma variedade de softwares semelhantes, em linguagens como **Java**, **R**, **Octave** e **MATLAB**. Linguagens como as citadas, são tipicamente utilizadas por pesquisadores da área, pelo fato de já possuírem uma gama de algoritmos destinados à análise de dados já implementados.

Visto que o principal motivo para o uso das linguagens citadas é a existência de bibliotecas de algoritmos famosos já implementados, o Python foi escolhido para o desenvolvimento deste trabalho. Como já apontado anteriormente, o Python possui uma vasta biblioteca, com uma comunidade de usuário ativa e crescente. Além disso, por possuir uma sintaxe simples, ela é ideal para que os pesquisadores, e futuros usuários, possam criar módulos capazes de estender as funcionalidades do projeto.

Por fim, a avaliação do software foi realizada com os pesquisadores. Eles propuseram a utilização e estudo de uma base de dados para avaliar a capacidade do sistema, de modo que seja satisfatório o problema proposto por ela.

3.3 Atividades

3.3.1 Cronograma de Atividades

A seguir é apresentado o cronograma de atividades do projeto, elaborado para que o mesmo fosse concluído no prazo, atendendo aos requisitos necessários e com a qualidade requisitada.

Quadro 1 – Cronograma de Atividades

Atividades	**Jul**	**Ago**	**Set**	**Out**	**Nov**	**Dez**
Entrevista com os Pesquisadores	X					
Pesquisa Bibliográfica		X				
Levantamento de Requisitos			X			
Documentação da Pesquisa				X		
Revisão do TCC I					X	
Defesa do TCC I						X

Fonte: O Próprio Autor (2022)

Capítulo 4
Desenvolvimento da Pesquisa

A pesquisa foi desenvolvida em duas etapas, a primeira corresponde a entrevista realizada com os pesquisadores do projeto *SmartEnergy* para o Levantamento de requisitos; a segunda etapa corresponde a pesquisa bibliográfica realizada para a elaboração do TCC I, o presente trabalho.

4.1 Levantamento de Requisitos

A etapa de levantamento de requisitos foi realizada em conjunto com os pesquisadores apontados anteriormente, com o intuito de identificar as principais funcionalidades e carências para uma plataforma do tipo proposto.

O levantamento ocorreu por entrevistas informais, onde os pesquisadores foram questionados sobre as funcionalidades que a plataforma deveria ter, e as carências que a plataforma deveria suprir. Foram apresentados base de dados, identificados os objetivos e então proposto abordagens que poderiam ser seguidas para atingir a solução.

4.1.1 Requisitos Funcionais

Os requisitos funcionais foram identificados a partir das funcionalidades que a plataforma deveria possuir, sido apresentadas aos pesquisadores após o levantamento. A seguir, são apresentados os requisitos funcionais identificados:

1. **Importação de Dados**: A plataforma deverá conseguir importar dados em diferentes formatos, como, por exemplo, arquivos CSV, JSON, XLXS, dentre outros.
2. **Pré-processamento de Dados**: A plataforma deverá conseguir realizar o pré-processamento dos dados, como, por exemplo, a normalização dos dados, remoção de dados faltantes, entre outros.
3. **Visualização de Dados**: A plataforma deverá conseguir apresentar os dados visualmente em gráficos, em barras, gráficos de dispersão, entre outros.
4. **Processamento de Dados**: A plataforma deverá ser capaz de realizar o processamento dos dados, como, por exemplo, a aplicação de algoritmos de aprendizado de máquina, ex.: regressão linear, regressão logística, entre outros.

5. **Exportação de Dados**: A plataforma deverá conseguir exportar os dados em diferentes formatos, como, por exemplo, arquivos CSV, JSON, XLXS, dentre outros.

6. **Métricas**: A plataforma deverá ser capaz de apresentar métricas de avaliação dos resultados, como, por exemplo, a acurácia, a precisão, a revocação, entre outros.

7. **Extensibilidade**: A plataforma deverá ser capaz de ser estendida para novos algoritmos a partir de interfaces de código tipado e aberto.

8. **Interface de Usuário**: A plataforma deverá ser capaz de apresentar uma interface de usuário intuitiva e personalizável.

9. **Exportação de Modelos**: A plataforma deverá conseguir exportar os modelos de aprendizado de máquina em binários com extensão.pkl e arquivos descritivos com extensão .json.

10. **Exportação de Configurações**: A plataforma deverá ser capaz de exportar as configurações de execução em arquivos com extensão .json.

11. **Personalização de Configurações**: A plataforma deverá ser capaz de permitir a personalização das configurações de execução, como, por exemplo, a quantidade de iterações, a quantidade de camadas, entre outros.

12. **Extensão da Interface**: A interface da plataforma deverá ser capaz de ser estendida para novas funcionalidades, preservando os estilos e funcionalidades já existentes.

4.1.2 Requisitos Não Funcionais

Os requisitos não funcionais, características que a plataforma deve possuir, mas que não são funcionalidades que o usuário precisa estar ciente, por se tratar de questões básicas de software. A seguir, são apresentados os requisitos não funcionais identificados:

1. **Execução Multiplataforma**: A plataforma deverá ser capaz de ser executada em diferentes plataformas, como, por exemplo, Windows, Linux, Mac OS, entre outros.

2. **Consumo de Recursos**: A plataforma, principalmente parte que não trata do processamento, deverá conseguir consumir poucos recursos de hardware, como, por exemplo, memória RAM, CPU, entre outros.

3. **Confiabilidade**: A plataforma deverá ser capaz de ser executada sem falhas, e em caso de falhas, deverá conseguir ser executada novamente sem a necessidade de intervenção humana.

4. **Desempenho**: A plataforma deverá ser capaz de ser executada em tempo hábil, sem a atrasar o processo de análise de dados.

5. **Status Operacional**: A plataforma deverá ser capaz de apresentar a situação de operação, como, por exemplo, se está processando os dados, salvando alterações, baixando dados, exportando gráficos ou configurações, entre outros.

6. **Log de Execução**: A plataforma deverá conseguir apresentar um log de execução, onde serão apresentados os eventos que ocorreram durante a execução, como, por exemplo, a importação de dados, a exportação de gráficos, a exportação de modelos, entre outros.

7. **Isolamento**: A plataforma deverá ser capaz de ser executada em um contêiner, como, por exemplo, Docker, para ser possível a execução em diferentes ambientes.

4.2 Proposta de Solução

A seguir é elaborado o conjunto de soluções para os requisitos levantados na seção anterior e que serão implementados no sistema. As soluções levantadas são exemplos de possibilidades, baseadas em estudos de caso e em experiências anteriores. A escolha da solução final será realizada durante a fase de desenvolvimento do sistema, mediante a análise de custo-benefício de cada uma das soluções.

4.2.1 Interfaces de Código

Uma interface de código pode ser descrita como um conjunto de instruções que permitem estabelecer a conexão entre duas bibliotecas ou até mesmo para promover o uso de uma biblioteca. Logo, o momento da definição da interface é uma das fases mais importantes da criação de um software. [REVISAR]

Para atender os requisitos da extensibilidade, através da pesquisa bibliográfica, as interfaces do projeto seguirão próximas as interfaces da maior biblioteca de *Machine Learning*, o *scikit-learn*. A biblioteca do *scikit-learn* é umas das mais extensas em número de implementações de algoritmos de aprendizado de máquina, além de possuir uma comunidade bastante ativa. Isso garante que além da compatibilidade, não só com a própria plataforma, como simplifica a extensão das funcionalidades do software.

4.2.2 Interface Gráfica

Para o desenvolvimento deste trabalho foi selecionado com a biblioteca baseada em Qt. O Qt provem então uma série de implementações voltadas a interface gráfica de usuário, por exemplo, a criação de janelas, diálogos, entre outros. A biblioteca fornece compatibilidade com a maioria dos sistemas operacionais atuais, o que atende ao requisito não-funcional da compatibilidade como diversos sistemas operacionais.

4.2.2.1 Widgets

O *PyQt* é uma biblioteca baseada em *widgets*, onde cada *widget* corresponde a um componente exibido na janela. Os *widgets* funcionam como *wrapper's*, funções que encapsulam outra chamada, diretamente para o objeto do Qt. Os *widgets* do Qt são divididos em duas categorias: os *widgets* de controle e os *widgets* de exibição. *Widgets* de controle são aqueles que permitem a interação do usuário com o sistema, como, por exemplo, botões, caixas de texto, entre outros. Já os *widgets* de exibição são aqueles que exibem informações ao usuário, como, por exemplo, tabelas, gráficos, entre outros.

Os *widgets* do *PyQt* são criados a partir de classes, são facilmente extensíveis, logo é possível reutilizar e criar componentes para o sistema. A criação de novos *widgets* é feita através da herança, onde uma classe é criada a partir de outra, herdando seus atributos e métodos. A herança é uma das principais características da orientação a objetos, que permite a reutilização de código, reduzindo a quantidade de código a ser escrito.

As ações no *PyQt* funcionam em um *event loop*, onde o sistema fica em um loop infinito, aguardando por um evento. Quando um evento é disparado, o sistema sai do loop e executa a ação correspondente ao evento. Como interfaces gráficas de usuário são orientadas a eventos, a biblioteca opera por eventos e sinais, para isso ele utiliza o conceito de *slots*. Os *slots* são funções que recebem um objeto de *callback*, objetos ou funções utilizadas para executar ações após um evento, para realizar uma tarefa quando um evento acontece e um sinal é enviado.

Por exemplo, em sistema com um botão e um contador embaixo, sempre que o usuário clica no botão, o contador é incrementado. O botão envia um sinal para o slot do contador, que incrementa o contador. O sistema fica em um loop infinito, aguardando por um evento, ao clicar no botão, o sistema sai do loop e executa a ação correspondente ao evento, incrementar o contador.

Logo, a orientação de eventos permite que o sistema seja responsivo para o usuário, atendendo as suas ações e apresentando uma resposta visual. No projeto, essa abordagem deve atender a praticamente todas as interações do usuário para configuração dos algoritmos de aprendizado de máquina e aos métodos de visualização.

4.2.2.2 Estilização

Para solucionar a questão dos estilos padrão para toda a interface e suas extensões, será utilizado a estilização por Cascading Style Sheets (CSS), a qual é uma linguagem de estilização que funciona a partir de uma série de regras que definem como os elementos da interface devem ser exibidos. A estilização por CSS permite a criação de estilos personalizados para cada elemento da interface, através dos seletores de elemento e de classe, como, por exemplo, um botão com identificador *#BackButton*, ou a estilização de caixa de texto com a classe *.Entrada*,

entre outros.

Outra possibilidade é a utilização de estilos pré-definidos, como, por exemplo, o *QDarkStyle* já presente na biblioteca, que fornece um estilo escuro para a interface, podendo ser utilizado para melhorar a experiência do usuário, principalmente em ambientes com pouca iluminação. Além disso, também é possível utilizar bibliotecas próprias para estilização da interface, que já dispõe de estilos conhecidos como o *Material Design*, a qual é um estilo de interface que utiliza elementos como, por exemplo, sombras, bordas arredondadas, entre outros.

A estilização é uma etapa importante para a criação de uma interface agradável para o usuário e deve ser feita de forma que atenda às necessidades do usuário. Usar cores facilmente reconhecíveis com contraste, saturação e brilho suficientes, além de usar elementos reconhecíveis, como botões com bordas arredondadas, etc. Tudo isso citado oferece aos usuários uma interface intuitiva e agradável.

4.2.2.3 Gráficos

Gráficos são uma das principais formas de visualizar dados e são uma das formas mais comuns de apresentar informações aos usuários. A biblioteca *Matplotlib* e sua extensão *Seaborn* são usadas para criar gráficos exibidos na interface via *widget MplWidget*. Este *weidget* é uma classe herdada de *FigureCanvas* e *NavigationToolbar*, classes da biblioteca *Matplotlib* responsáveis por criar o gráfico e a barra de ferramentas, respectivamente.

Matplotlib é uma biblioteca gráfica que permite a criação de gráficos 2D e 3D, além de gráficos animados. Esta biblioteca é usada para criar gráficos de linhas, gráficos de barras, histogramas, etc.

Já a biblioteca *Seaborn* é uma extensão do *Matplotlib* que permite a criação de gráficos melhores, com melhores estilos e cores, e, além disso, permite a criação de gráficos mais complexos, como gráficos de dispersão, intervalo de gráficos de linha confiável e assim sobre. A biblioteca *Seaborn* deve atender as necessidades de visualização de dados do projeto, pois permite a criação de gráficos mais complexos e com melhor estilização, além de que poderá ser facilmente extensível para a criação de novos gráficos.

4.2.3 Gerenciamento de Dados

Para solucionar a questão de gerenciamento de dados, será utilizado o *Pandas* para a manipulação dos dados, sendo uma biblioteca de código aberto que fornece estruturas de dados e ferramentas de análise de dados para a linguagem de programação Python. O *Pandas* é uma biblioteca que permite a manipulação de dados, como, por exemplo, a leitura de arquivos, a criação de tabelas, a criação de gráficos, entre outros. A biblioteca *Pandas* é uma das bibliotecas

mais utilizadas para manipulação de dados, pois permite a manipulação de dados de forma simples e eficiente, além de que possui uma grande comunidade de usuários e desenvolvedores.

O *Pandas* utiliza o núcleo do *Numpy*, a qual é uma biblioteca de código aberto que fornece suporte para *arrays*, vetores, e matrizes multidimensionais, além de funções matemáticas para operações com esses *arrays*. A performance do *Numpy* é devido ao uso da linguagem C para a criação e operação com arrays, garantiu uma grande velocidade de processamento, além de que o *Numpy* é uma biblioteca amplamente utilizada em outras bibliotecas, como, por exemplo, o *Tensorflow*.

Utilizando o *Pandas* é possível importar e exportar tabelas facilmente, já que a biblioteca já possui esses métodos. Além disso, também é possível realizar operações como ordenação, filtragem, entre outras tarefas básicas da análise de dados.

Por fim, com esta proposta de solução, a plataforma conseguirá atender a maioria dos requisitos funcionais e não funcionais apresentados anteriormente. O projeto deve ser desenvolvido utilizando a linguagem de programação *Python*, a qual é uma linguagem de alto nível, interpretada, de tipagem dinâmica e multi-paradigma.

A linguagem *Python* possui uma grande comunidade de desenvolvedores, facilitando a obtenção de ajuda e suporte, além de ser uma linguagem de fácil aprendizado.

A plataforma será desenvolvida utilizando as interfaces da biblioteca de aprendizado de máquina *Scikit-learn*, a qual é uma biblioteca de código aberto, de código tipado, que possui uma grande comunidade de desenvolvedores e que possui uma grande variedade de algoritmos de aprendizado de máquina.

O desenvolvimento da plataforma será a partir a biblioteca de visualização de dados *Matplotlib*, sendo uma biblioteca de código aberto, de código tipado, que possui uma grande comunidade de desenvolvedores e que possui uma grande variedade de gráficos para visualização de dados.

A interface gráfica de usuário será desenvolvida utilizando a biblioteca *PyQt*, a qual é uma biblioteca de código aberto, de código tipado, que possui uma grande comunidade de desenvolvedores e que possui uma ampla variedade de componentes, ou *widgets*, para a construção de interfaces gráficas de usuário.

Capítulo 5
Resultados

Com a proposta elaborada neste trabalho, é esperado a obtenção de uma plataforma capaz de integrar facilmente os dados; que seja possível estender as funcionalidades para vários algoritmos de aprendizado de máquina; além claro, da extensão da própria interface, tudo isso para que a plataforma atenda a demanda de diversos tipos de problemas.

Em relação aos dados, a plataforma deverá oferecer a suporte a diferentes fontes de arquivos, tanto arquivos locais, quanto arquivos em nuvem, como, por exemplo, o Google Drive. Além disso, a plataforma deverá conseguir integrar dados de diferentes tipos, como, por exemplo, dados de um banco de dados relacional, dados de um aquivo de planilha como XLXS e dados de um arquivo CSV.

A plataforma também deverá integrar novos algoritmos a partir das interfaces herdadas do *scikit-learn*, como, por exemplo, o *Random Forest* e o *Gradient Boosting*. Além disso, a plataforma deverá ser capaz de integrar novos algoritmos de aprendizado de máquina, como, por exemplo, o *Support Vector Machine* e o *Neural Network*, com a condição da utilização das interfaces compatíveis. Desta forma será possível integrar novos algoritmos com poucas linhas de código.

A interface, no que lhe concerne, deverá ser capaz integrar novas funcionalidades, como, por exemplo, a criação de novos tipos de gráficos, como, por exemplo, gráficos de dispersão, gráficos de barras e gráficos de pizza. Além disso, a interface deverá conseguir integrar novas funcionalidades, como, por exemplo, a integração de novos algoritmos com parâmetros personalizados, como, por exemplo, o algoritmo *KNN* com o parâmetro *weights*.

Desta forma, a plataforma deverá ser capaz de atender a demanda de diversos tipos de problemas, como, por exemplo, problemas de classificação, problemas de regressão e problemas de agrupamento. Seguindo os requisitos propostos, a plataforma deverá conseguir atender a demanda levantada no começo do trabalho com os pesquisadores do *SmartEnergy*.

Capítulo 6
Conclusão

A seguir serão apresentadas as conclusões do trabalho, as contribuições e limitações do mesmo, e os trabalhos futuros.

6.1 Discussão

O presente trabalho apresentou uma proposta de uma plataforma de análise e visualização de dados a ser desenvolvida em Python, atualmente uma das linguagens de programação mais utilizadas no mundo. A plataforma tem por objetivo ser facilmente extensível, permitindo a adição de novos algoritmos e etapas no processo de análise de dados, além de uma interface que possa ser personalizada e modificada de forma simples.

A plataforma teve os requisitos levantados junto a um grupo de pesquisadores da área de Ciência de Dados e Processamento de sinais, que participaram do projeto *SmartEnergy*, esse sendo a principal inspiração para este projeto. A plataforma do *SmartEnergy* foi desenvolvida em Python, durante o projeto, mas não possuía os mesmos requisitos que os levantados aqui. Além disso, a plataforma já existente não foi projetada seguindo padrões de projeto, o que dificultou a extensão e a manutenção do código.

Logo, a plataforma proposta no presente trabalho foi pensada para atender aos requisitos levantados, suficiente para atender as necessidades do projeto *SmartEnergy*, e também para ser utilizada em outros projetos com problemas distintos.

6.2 Contribuições

O projeto desenvolvido neste projeto necessitou de uma extensa pesquisa bibliográfica, para ser possível identificar as melhores ferramentas e técnicas para a implementação da plataforma. O projeto considerou uma série de possibilidades de extensão e com foco na manutenção continua, não somente pelos desenvolvedores, mas também através da comunidade de desenvolvedores Python. Embora a arquitetura não esteja definida nesta etapa do trabalho, o projeto deverá seguir um conjunto de padrões de projeto e boas práticas de programação, para poder ser facilmente mantido e evoluído.

Logo, o projeto contribuiu como um estudo de caso do desenvolvimento do projeto de uma plataforma de análise de dados, que pode ser utilizada como referência para plataformas com

finalidades específicas. Além disso, o projeto contribuiu com a comunidade de desenvolvedores Python, com a proposta de uso do conjunto de interfaces e bibliotecas apresentadas anteriormente.

6.3 Limitações

O trabalho apresentado possui algumas limitações, questões naturais deste tipo de projeto, como a alta variedade de tipos de dados, algoritmos e possíveis problemas que a plataforma poderá auxiliar na resolução. As limitações estão dispostas a seguir.

Devido à grande variedade de algoritmos, é possível que no começo do desenvolvimento do projeto, a integração com alguns algoritmos não seja simples de tal modo como o esperado para o *scikit-learn*.

Outro ponto é que o processo de análise em *pipeline*, a organização da aplicação dos algoritmos, não será simples, devido à grande variedade de interfaces que cada algoritmo e problema dispõe. Entretanto, é possível que durante a próxima etapa do projeto, seja possível identificar uma forma de organizar os algoritmos conforme as necessidades de cada problema abordado pelos dados.

Além disso, é possível que o desenvolvimento do projeto não seja tão simples quanto o esperado, pois o mesmo é um projeto de grande porte, que envolve diversas tecnologias e ferramentas. De todo modo, o projeto seguira o planejamento estabelecido, e será desenvolvido consoante os requisitos levantados.

6.4 Trabalhos Futuros

Para o futuro deste projeto é esperado que o mesmo seja desenvolvido segundo o planejamento estabelecido. Entretanto, é possível que durante o desenvolvimento do projeto, seja necessário realizar algumas mudanças no planejamento, devido a dificuldades encontradas durante o desenvolvimento.

Referências

GALVãO, Z. J. A. *Gerenciamento Inteligente do Consumo de Energia*. 2020. Disponível em: <http://www.cic.propesq.ufrn.br/trabalhos.php>. Acesso em: 26 de dezembro de 2022. Citado na página 16.

APÊNDICE A

Cronograma de Atividades